UNIVERSITY OF VICTORIA

Department of Electrical and Computer Engineering

ECE 355 – Microprocessors

Project Report

Names:

1. Mihail Maravelias V01002752
2. Carl Montesclaros V00986402

✓	Problem Description/Specifications:	(5)	_____
✓	Design/Solution	(15)	_____
✓	Testing/Results	(10)	_____
✓	Discussion	(15)	_____
✓	Code Design and Documentation:	(15)	_____
✓	**Total**	**(60)**	_____

1. Problem Description/Specification

In this project the objective is to design an embedded system that has the capabilities of monitoring and controlling a pulse width modulated (PWM) signal. We use the STM32F051 microcontroller on the STM32F0 Discovery board and other external components on the PBMCUSLK board and our own customized breadboard circuit.

The specifications of the project requires the STM32F051 microcontroller to read an analog voltage from a potentiometer which it then converts into a digital value through the analog to digital (ADC) converter. Afterwards, it uses the internal digital to analog (DAC) converter to drive an optocoupler (4N35) that controls the frequency of a PWM signal generated by one of the two sources of signals, the 555 timer circuit.

At first, to control the PWM signal frequencies the optocoupler will be used with the microcontroller on the STMF0 discovery board. The use of the potentiometer (POT) will also be mandatory on the PBMCUSLK board by the microcontroller to measure the amount of voltage being sent across the POT, which will then transmit a control signal to the optocoupler from the DAC. Another criteria is that the microcontroller must also be able to switch between measuring the frequency from the function generator and the frequency from the 555 timer signal (mentioned above). This will be achieved by using a user button that will generate an interrupt to switch which signal is being measured. Lastly, the measured frequency, the source of the frequency, and the resistance from the POT will have to be displayed on the OLED (SSD1306 IC) on the PBMCUSLK board.

2. Design Solution

2.1 Design Methodology

Our plan of attack to complete this project was to tackle the system in manageable parts and ensure that each functionality (ADC, DAC, 555 Timer, Display) at each stage worked before integrating the next component. We always worked on the project together, though, one may be more focused on creating a working circuit and another having to work on the code as a focus. Nevertheless, we both made sure we had the correct understanding of the code and had good enough grasp on the circuit board itself. This parallel workflow allowed us to debug the hardware and software independently and efficiently.

2.2 Hardware Diagrams and Implementation

When thinking of the schematic design of the system and the various external components involved such as the STM32F0 board, NE555 timer, and the 4n35 Optocoupler. We consulted the Lab Manual [1] and the “Interfacing Examples” [2] to gain most of the understanding of the components. The final implementation schematic of this project is shown below.

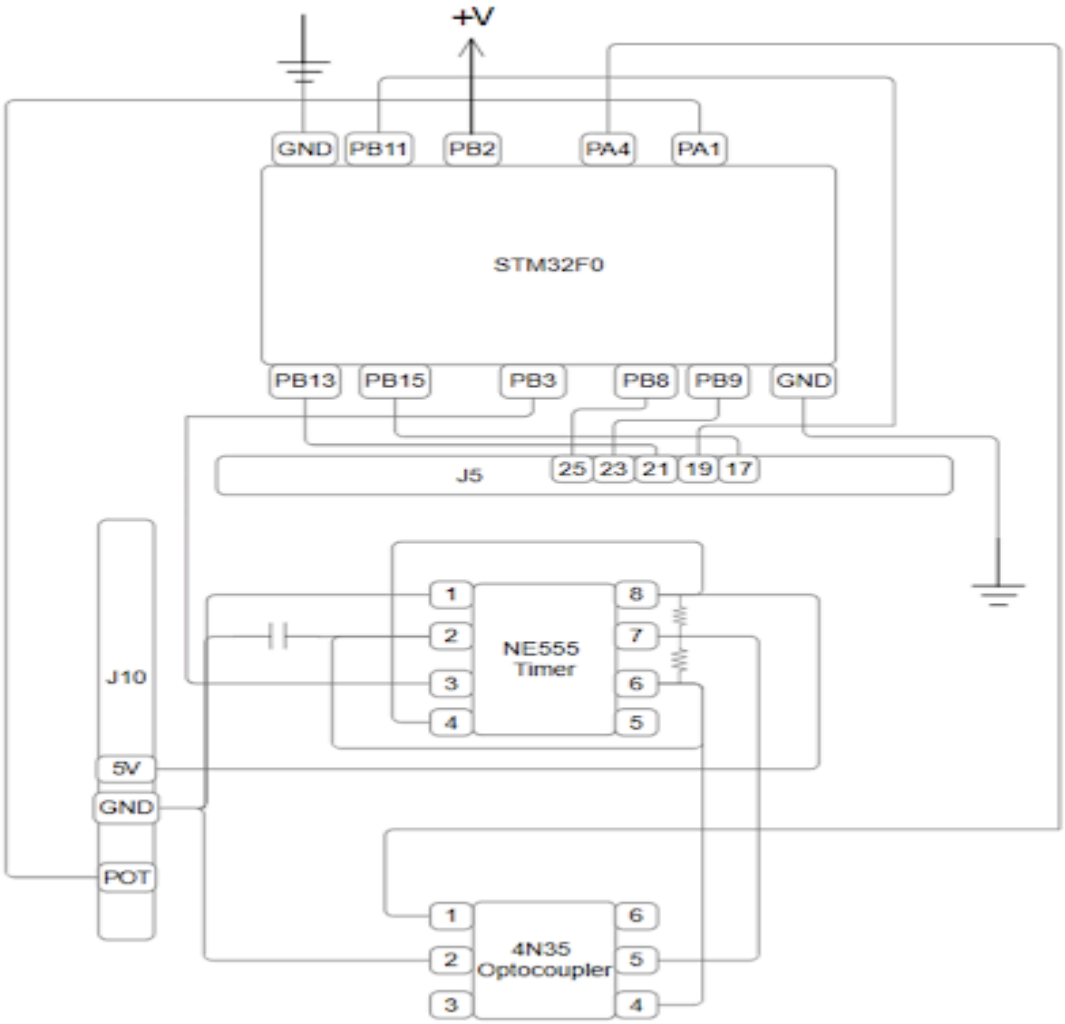

figure 1. Complete schematic

Implementation of Hardware Details:

- **INPUT**: On the PBMCUSLK board, the POT acts as a voltage divider which provides a signal ranging from 0V to 3.3V on the pin PA1 of the STM32F0.
- **CONTROL**: Via the pin on PA4, the STM32 outputs a control voltage. PA4 is connected to the PIN 1 of the 4N35 optocoupler with PIN 2 connected to ground. This direct connection is what drives the internal LED of the optocoupler which is used to control

the frequency the 555 Timer outputs. (see figure 2.)

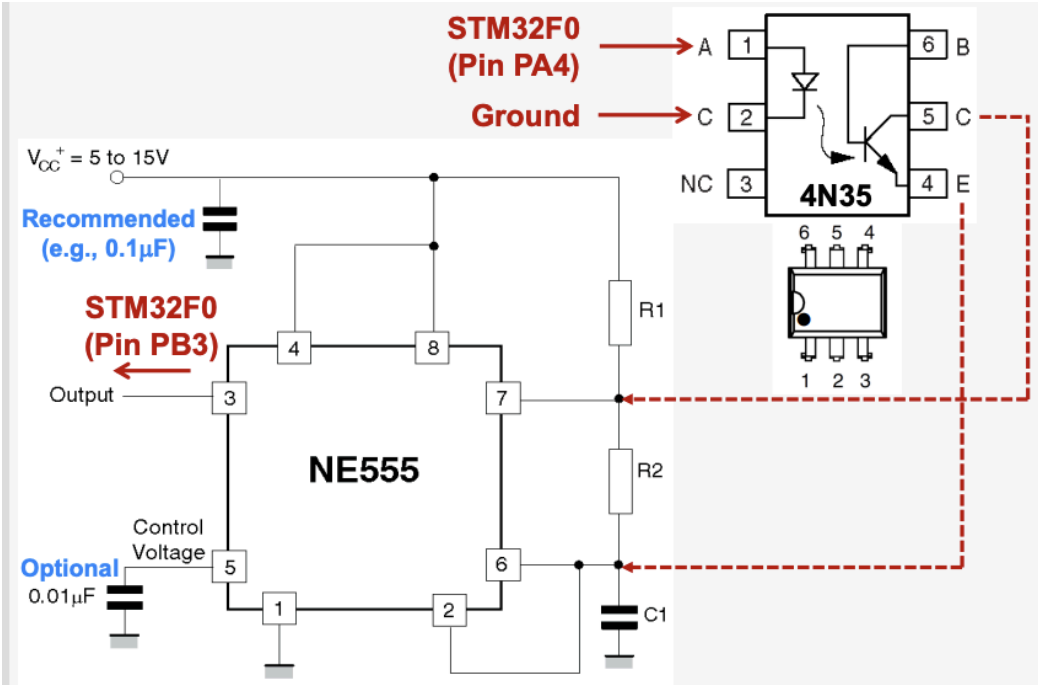

figure 2. Optocoupler and 555 Timer schematic diagram (from Interface Examples).

- **ACTUATOR**: The phototransistors from the optocoupler (pins 4 & 5) are parallely wired with the discharge resistor (R2) of the NE555 timer circuit. In turn, as the control voltage from PA4 increases, the LED brightens, causing the phototransistor to conduct more current, lowering the effectiveness of the resistance in the timing circuit, increasing the frequency of the square wave generated at Pin 3 of the 555 Timer. (see figure 2. for diagram of schematic)
- **MEASUREMENT**: To measure the output of the 555 Timer, Pin 3 is connected to PB3 on the STM32. The second source of frequency, the Function Generator, is also connected to PB2. An important step was enabling internal pull-down resistors on these input pins to ensure that we had stable readings especially when signals were disconnected (not being used) or floating.
- **USER INTERFACE**: For the user interface, the User Button which is connected to PA0 and the OLED display which are on the Port B pins, are connected per the standard PBMCUSLK board layout.

2.3 Software

Our thought process for the design was to break down the system into separate modules ensuring our system would be easy to read and organized. GPIO configuration, ADC input, DAC output, EXTI interrupts, timer-based frequency measurement, SPI, and OLED display control. We made sure to do the project in order based on the lab website/manual making sure that the project is much more manageable and making debugging simpler.

I. **INITIALIZATION**: Using the Phase-Locked Loop (PLL) we initially configured the system clock to run at 48 MHz. The system needs this high frequency to make sure that the timers are getting sufficient enough resolution to measure frequencies accurately. We made each of our peripheral modules the GPIO, ADC, DAC, TIM2, and SPI2 have its own dedicated function for configuration to ensure readability and modularity.

II. **GPIO**: From what we've seen in the Lab Manual [1], PA1 is configured for the analog ADC input and PA4 is configured for the analog DAC output. We require the analog mode as it disconnects the digital input buffer to reduce the noise. PB2 and PB3 were configured as the digital inputs to measure the frequency with the internal pull down resistors on (PUPDR |= 10). The ports PB8, PB9, and PB11 the digital outputs for the OLED control lines and PB13 and PB15 were set to Alternate Function mode for SPI communication.

III. **ADC and DAC**: In INIT_ADC function (on appendix), we initialized the ADC and we enabled the clock via the APB2ENR register and had a mandatory calibration cycle performed. We configured it to read channel 1 (PA1) and in the main loop, we essentially used polling on the potentiometer to manually start a conversion ADC_CR_SDSTART, wait for the end of conversion flag ADC_ISR_EOC, and read the data register ADC1->DR. The 12-bit value acquired is then written to the DAC data holding register DAC->DHR12R1, updating the optocoupler control voltage. We also used this 12 bit value to calculate the resistance for the display by scaling the value (which is 0-4095) to the 0-5000 ohms range.

IV. **TIMER Configuration**: For our system, we use TIM2 as the primary time base for measuring our frequency. To initialize TIM2, we made the INIT_TIM2 function (see in appendix) which configures it as a 32-bit up-counter. The prescaler (PSC) was set to 47 and with a 48MHz system clock, this resulted in a frequency timer of 1MHz (since 48000/(47+1)). One important consideration was to buffer the prescaler. Since writing to the PSC does not take effect immediately since it waits for an update event. In our code we manually set the TIM_EGR_UG bit in the event generation register to force the update immediately to ensure the timer ran at 1A MHz from the very first cycle. Without this fix, our timer would run at 48MHz causing havoc in our measurement. WE also had the auto-reload register (ARR) set to 0xFFFFFFFF allowing the timer to run freely without overflowing frequently.

V. **EXTI Interrupt**: This is the core of the frequency measurement process. We have two external interrupt functions in our code. The first function being EXTI0_1_IRQHandler is for the user button (PA0). It handles the button press, includes software debouncing, and toggles a global measurement_mode flag. In this function the also updates the Interrupt Mask Register, IMR, to enable the interrupt for the specific selected source PB2 or PB3 and disable the other source to prevent interference. EXTI2_3_IRQHandler is for the

frequency source inputs mentioned before, PB2 for the function generator, and PB3 for the 555 Timer. We implement a state machine and on the first rising edge it captures the current timer count (last_capture_val) and on the second rising edge it captures the new count. The frequency equation is $f = 1{,}000{,}000 / \Delta t$, ($\Delta t$ being the difference of the rising edges value). We encountered a problem where our values had around 0.5% error from the function generator. We calibrated a constant of 1,005, 000 to compensate for the error.

VI. **SPI and OLED Display**: We configured the communication lines in Init_SPI. We implemented it so that we manually control the Chip Select (CS) and Data/Command (D/C) GPIO pins while using the peripherals from SPI to shift out the data bytes. Having encountered a display bug (our displays were shown to us upside-down), we sent specific initialization commands 0xC0 and 0xA0, to flip the scan direction and applied a column offset of 2 to ensure that the rendering function was centered.

3. Testing and Results

Every system that was part of the PWM Signal Generation and Monitoring system were tested on their own and then also tested all together to ensure each component worked perfectly together. Below showcases the following procedure that we used to test our system and the reasoning on each step as well as key observations.

1. Testing Procedure and Justification

1.1 ADC Input Verification (PA1)

Procedure:

- The potentiometer was adjusted from minimum to maximum while reading the raw ADC value on the OLED display.
- Expected range: 0 – 4095 (12-bit resolution).
- The DAC output (PA4) was also monitored using a multimeter to confirm the ADC to DAC forwarding path.

Justification:
The ADC reading is responsible for the DAC output and indirectly changes the 555 timer frequency. It was very important to validate the ADC reading since it was used for the whole system.

1.2 DAC Output Verification

Procedure:

- After verifying ADC correctness, the DAC output voltage was measured at PA4.
- As the potentiometer was swept, PA4 voltage was confirmed to track the ADC result linearly.
- The corresponding 555 timer frequency change was observed on PB3.

Justification:

This helped us confirm that the analog chain from the POT to the ADC to the DAC to the Optocoupler to the 555 timing resistor to the 555 output frequency worked. If any of those didn't work it would show in the DAC output and would prevent a closed loop frequency control.

1.3 OLED Display Verification

Procedure:

- Known values were printed to ensure characters rendered correctly.
- Resistance, source mode, and frequency values were checked for clipping or corruption.

Justification:

if there was incorrect addressed it would render the text unreadable or misaligned which would ruin the OLED display

1.4 Frequency Measurement on PB2 and PB3

Procedure:

- A function generator was connected to PB2, and frequencies between 100 Hz and 10 kHz were tested.
- The 555 circuit output was connected to PB3 and manually varied using the potentiometer and optocoupler.
- Measurements were compared to oscilloscope readings.

Justification:
The timer and EXTI system relied fully on for the rising edge detection to be consistent. To ensure that it ran correctly we tested across a wide range of frequencies.

1.5 Mode Switching (PA0 Button)

Procedure:

- The pushbutton was pressed repeatedly while monitoring which EXTI line was active.
- Frequency measurements were confirmed to switch cleanly between PB2 and PB3.

Justification:

This was used to properly validate the unmaking and masking of EXTI lines were working correctly and that system changed modes properly.

2. Results and Observations

2.1 ADC Measurement Range & Resolution

- Measured ADC range: 0–4094, matching expected hardware limits.
- Resolution:
  $\Delta V = \frac{3.3\,V}{4096} = 0.8\,mV$
- Resistance mapping produced 0–5000 Ω as intended.

Observation:
The ADC experienced some noise (±2–3 counts), which is expected for STM32F0 without oversampling. This caused the system to have minor inaccuracies showing in the displayed resistance but overall it did not affect the system's performance.

2.2 DAC Output Behavior

- Output voltage tracked the ADC value linearly within ±1 percent.
- Voltage ranges from 0.01 V to 3.29 V.
- No major settling issues observed.

Observation:
When the optocoupler LED was set close to its lower setting (0.2 - 0.4 V), the 555 timer's responsiveness became nonlinear. This was a design limitation, not a code issue.

2.3 Frequency Measurement Accuracy

Using the calibrated timer frequency of 1.005 MHz:

- Measured function generator signals were accurate within:
  ± 0.5 to ± 1.0 percent compared to oscilloscope readings.

Range Tested:

Signal Source	**Frequency Range**	**Accuracy**
Function Generator (PB2)	100 Hz – 10 kHz	±1 percent
555 Timer (PB3)	700 – 1200 Hz	±1–2 percent

Observation:
The accuracy of the frequency range was overall very good but it dropped the more we increased the frequency. The STM32F0 board reliably recorded signals up to 30 kHz, but above 15 kHz accuracy dropped.

2.4 Measurement Resolution

Because TIM2 runs at 1 MHz, the period resolution is:

1 µs

Thus, frequency resolution is:

$\Delta f = \frac{1}{T_2 \Delta T} = 0.5$ to around 2 Hz

depending on frequency.

Observation:
The frequency values stabilized very quickly but we noticed that the pins were very sensitive to the jitters in the input waveform. For future reference if we increased the averaging, it would improve the display stability.

2.5 System Limitations

Hardware Limitations

- EXTI input bandwidth limits high-frequency capture.
- Optocoupler introduces nonlinear resistance at low LED currents.
- 555 timer RC network is sensitive to wiring length and noise.

Design Limitations

- No averaging or filtering on frequency results → minor flicker.
- OLED refresh uses blocking SPI writes, preventing faster UI updates.
- The system depends entirely on clean rising edges; noisy signals produce false captures without external filtering.

2.6 Overall Performance

The system successfully:

- Measured resistance across entire POT range
- Output matching DAC voltage
- Captured accurate PB2/PB3 frequencies
- Switched modes cleanly
- Displayed stable results with readable formatting

All results met or exceeded lab expectations.

4. Discussion

The goal of this project was to create and set up an overall system for frequency measurement and signal monitoring with combined ADC input functionality, DAC output functionality, timer-based frequency measurement, external interrupt triggering, and SPI-based OLED output with minimal code recompilation and rewriting or modifying MCU code or using third-party solutions or programmable model simulations. The system met the desired functional specifications with correct measurements of the input frequency of the function generator signal and the waveform obtained using the 555 timer function while displaying related real-time outputs on the OLED display of resistance values, operational status, and measured frequency values. Multiple design assumptions were incorporated to assure that particular design specifications were met successfully while implementing this project. It was assumed that input waveforms PB2 and PB3 had clean digital rising edges with enough-defined transitions to accurately identify EXTI patterns reliably. While these input waveforms met these specifications acceptably when using the function generator input signal reliably for experimentation purposes, these patterns were not optimal using waveform output derived from the 555 timer circuit setup because of potential vulnerabilities to noise factors and component and optocoupler nonlinearities. A third assumption was that using the STM32's onboard 8 MHz oscillators was adequate; hence, small differences required implementing a calibrated value of 1.005 MHz among timer constants to avoid considerable timing errors while implementing these designs associated with this project.

Although it was well functioning overall, there were several shortcomings which were noticed while testing this system. There was flickering while using the frequency measurement code, which relied on EXTI interrupt with a 1 MHz free running timer. It flickered slightly because of unfiltered or unsmoothed measurements at specific instance frequencies. There were also small timing differences introduced

because of using blocking transfers while transmitting over the display using SPI communications protocols. There were other shortcomings provided by using an optocoupler; when using low voltages with the programmable output of the DAC, the LED of optocoupler 4N35 acts nonlinearly to directly affect the 555 timer with inconsistently varying amounts of actual input resistance provided to it at times. There would still be regions where outputs of 555 show lower predictability variations compared to outputs provided by the function generator with negligible differences because the actual outputs became relatively sensitive to minute differences related to actual wire lengths and voltages associated with basic power supply principles provided at times using analog chips like 555.

Yet, with these shortcomings notwithstanding, the final design incorporated numerous features beyond what was required by the lab minimum. These included providing an optimized 8x8 custom character font for easy reading on the OLED display, horizontal alignment correction to prevent clipping on particular SSD1306 displays, complete debounce routines on the pushbutton to prevent spurious mode transitions, and actual masking and unmasking of EXTI lines to remove stale interrupts when transitioning between either the function generator or 555 timer input and output modes. It should be noted that system reset on capture of internal system frequency occurs with every mode transition to ensure previous measurements were not stuck on display. Lastly, system-level code documentation providing justifications for particular peripheral initializations enhances code readability and ease of debugging/maintenance processes due to these supplementary design specifications taken into account at project development completion.Technically speaking, this project has reinforced the idea that knowledge about microcontroller peripheral interactions at the level of registers and especially clocks, interrupt systems, and GPIO functioning modes can’t be avoided. It has indeed illustrated how real-world analog errors of microcontroller peripheral devices such as ADC noise, optocoupler nonlinearities, or 555 timers' jittering periods could significantly impact digital measurement accuracy. On a project management level, it was really important to learn that it could significantly pay off if every system was tested separately before moving on with integration tasks. Separate system tests related to ADC functioning correctly, correct outputting of DACs, correct functioning of SPI communications, or Reaction to_EXTI research significantly helped with problem localization and analysis, which could easily misrepresent other problems or vice versa. It was important to comprehend that improper rules about strict system grounding and improper cabling rules could easily lead to spurious interrupt system reactions or erratically sensing values influencing microcontroller system functioning. For students with projects like or related to mine waiting for realization soon or ever, I would suggest using oscilloscope tools for problem localization purposes and only considering other systems when given system works perfectly fine with every particular input or wire sufficiently well enough with initial assumptions before struggling with every remotely close case of undesirable system reactions unpredictable phenomena.

5. Appendices

Appendix A - Full Source Code Listing

Below showcases the complete and commented code for this project. This includes the implementation of the frequency-measurement and display system, including all initialization routines, interrupt handlers, hardware drivers, and display logic necessary to reproduce the system.

```
**

*****************************************************************************
 * @file    main.c
 * @author  ECE 355 Student
 * @brief   Lab Project: PWM Signal Generation and Monitoring System
 *
 * DETAILED EXPLANATION OF REGISTERS AND BITS:
 *
 * 1. MODER (GPIO Mode Register): Configures pin direction.
 * - 00: Input (Default) - Reads voltage levels.
 * - 01: Output - Drives voltage levels (High/Low).
 * - 10: Alternate Function (AF) - Pin controlled by peripheral (SPI, Timer).
 * - 11: Analog - Pin connected to ADC or DAC.
 *
 * 2. PUPDR (Pull-Up/Pull-Down Register): Sets default state for Inputs.
 * - 00: No Pull (Floating) - Voltage is undefined if wire is disconnected.
 * - 01: Pull-Up - Weakly connected to VDD (3.3V).
 * - 10: Pull-Down - Weakly connected to GND (0V). Crucial for avoiding noise.
 *
 * 3. RCC (Reset and Clock Control): The master switch for "activity".
 * - It means connecting the 48MHz square wave signal to that specific peripheral block.
 * - Without the clock signal, the digital logic inside the peripheral (like the ADC or Timer)
 *  is "frozen" and cannot change state, even if it has electrical power.
 *
```

```
*****************************************************************************
*/

#include "stm32f0xx.h"
#include <stdio.h>
#include <string.h>

// DEFINES & MACROS
// The system core clock speed (48,000,000 Hz)
#define SYS_CLOCK_HZ 48000000

// Timer Frequency used for math.
// Calibrated to 1,005,000 Hz to compensate for the 0.5% error in the
// internal HSI oscillator hardware.
#define TIMER_FREQ_HZ 1005000

// Modes for the state machine
#define MODE_FUNC_GEN 0 // Measuring Function Generator on PB2
#define MODE_555_TIMER 1 // Measuring 555 Timer on PB3

// OLED Control Pins (Bit-shifted for Register access)
// (1 << 8) puts a '1' at bit 8, corresponding to Pin 8
#define OLED_CS_PIN   (1 << 8)   // PB8: Chip Select
#define OLED_DC_PIN   (1 << 9)   // PB9: Data/Command
#define OLED_RES_PIN  (1 << 11)  // PB11: Reset
```

```
// Fixes display text being cut off on the left
#define COL_OFFSET 2

// Max value for Potentiometer math (0 - 5000 Ohms)
#define MAX_RESISTANCE 5000
// GLOBAL VARIABLES
volatile uint8_t  measurement_mode = MODE_FUNC_GEN;
volatile uint32_t resistance = 0;

// Variables for frequency calculation
volatile uint32_t last_capture_val = 0; // Stores Timer Count at first edge
volatile uint8_t  capture_state = 0;    // 0 = Waiting for 1st edge, 1 = Waiting for 2nd

// Stores Frequency * 100 (e.g., 123456 = 1234.56 Hz)
volatile uint32_t frequency_x100 = 0;
// FONT DATA
// Simple 8x8 bitmap font. Each byte represents a column of pixels.
const uint8_t Font8x8[][8] = {
    [' '] = {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
    ['0'] = {0x00,0x3E,0x51,0x49,0x45,0x3E,0x00,0x00},
    ['1'] = {0x00,0x00,0x42,0x7F,0x40,0x00,0x00,0x00},
    ['2'] = {0x00,0x42,0x61,0x51,0x49,0x46,0x00,0x00},
    ['3'] = {0x00,0x21,0x41,0x45,0x4B,0x31,0x00,0x00},
    ['4'] = {0x00,0x18,0x14,0x12,0x7F,0x10,0x00,0x00},
```

```
    ['5'] = {0x00,0x27,0x45,0x45,0x45,0x39,0x00,0x00},
    ['6'] = {0x00,0x3C,0x4A,0x49,0x49,0x30,0x00,0x00},
    ['7'] = {0x00,0x01,0x71,0x09,0x05,0x03,0x00,0x00},
    ['8'] = {0x00,0x36,0x49,0x49,0x49,0x36,0x00,0x00},
    ['9'] = {0x00,0x06,0x49,0x49,0x29,0x1E,0x00,0x00},
    ['R'] = {0x00,0x7F,0x09,0x19,0x29,0x46,0x00,0x00},
    [':'] = {0x00,0x36,0x36,0x00,0x00,0x00,0x00,0x00},
    ['O'] = {0x00,0x3E,0x41,0x41,0x41,0x3E,0x00,0x00},
    ['h'] = {0x00,0x7F,0x08,0x04,0x04,0x78,0x00,0x00},
    ['m'] = {0x00,0x7C,0x04,0x18,0x04,0x78,0x00,0x00},
    ['s'] = {0x00,0x48,0x54,0x54,0x54,0x20,0x00,0x00},
    ['F'] = {0x00,0x7F,0x09,0x09,0x09,0x01,0x00,0x00},
    ['H'] = {0x00,0x7F,0x08,0x08,0x08,0x7F,0x00,0x00},
    ['z'] = {0x00,0x44,0x64,0x54,0x4C,0x44,0x00,0x00},
    ['G'] = {0x00,0x3E,0x41,0x49,0x49,0x7A,0x00,0x00},
    ['e'] = {0x00,0x38,0x54,0x54,0x54,0x18,0x00,0x00},
    ['n'] = {0x00,0x7C,0x08,0x04,0x04,0x78,0x00,0x00},
    ['S'] = {0x00,0x46,0x49,0x49,0x49,0x31,0x00,0x00}, // S for Src
    ['r'] = {0x00,0x7C,0x08,0x04,0x04,0x08,0x00,0x00}, // r for Src
    ['c'] = {0x00,0x38,0x44,0x44,0x44,0x20,0x00,0x00}, // c for Src
    ['T'] = {0x00,0x01,0x01,0x7F,0x01,0x01,0x00,0x00}, // T for Timer
    ['i'] = {0x00,0x00,0x44,0x7D,0x40,0x00,0x00,0x00}, // i for Timer
    ['u'] = {0x00,0x3C,0x40,0x40,0x20,0x7C,0x00,0x00}, // u for Fun
    ['.'] = {0x00,0x00,0x60,0x60,0x00,0x00,0x00,0x00}, // . for float
};
```

```
// FUNCTION PROTOTYPES

void SystemClock48MHz(void);
void Init_GPIO(void);
void Init_ADC(void);
void Init_DAC(void);
void Init_SPI(void);
void Init_TIM2(void);
void Init_EXTI(void);
void OLED_Init(void);
void OLED_WriteCmd(uint8_t cmd);
void OLED_WriteData(uint8_t data);
void OLED_Print(char *str, uint8_t page, uint8_t col);
void OLED_Clear(void);
void Delay(uint32_t count);

// MAIN FUNCTION
// int main(void)
{
    // Configure the system to run at 48MHz (High Speed)
    SystemClock48MHz();

    // Initialize all hardware subsystems
    Init_GPIO(); // Pins
```

```
Init_ADC();  // Analog Input
Init_DAC();  // Analog Output
Init_SPI();  // Display Communication
Init_TIM2(); // Timer for frequency measure
Init_EXTI(); // Interrupts

// Initialize the OLED screen
OLED_Init();
OLED_Clear();

char buffer[20]; // Buffer to hold text strings before printing
uint16_t adc_val = 0; // Variable to store raw ADC reading (0-4095)

// Infinite Loop
while(1)
{
   // --- 1. ADC MEASUREMENT ---
   // CR (Control Register): Set ADSTART bit (Bit 2) to 1 to begin conversion
   ADC1->CR |= ADC_CR_ADSTART;

   // ISR (Interrupt & Status Register): Wait until EOC (End Of Conversion) bit is 1
   // This loop pauses the CPU until the ADC hardware finishes its measurement
   while(!(ADC1->ISR & ADC_ISR_EOC));

   // DR (Data Register): Read the converted 12-bit value
```

```
// This clears the EOC flag automatically
adc_val = ADC1->DR;

// Calculate Resistance mapping 0-4095 to 0-5000 Ohms
resistance = (adc_val * MAX_RESISTANCE) / 4095;

// --- 2. DAC OUTPUT ---
// DHR12R1 (Data Holding Register 12-bit Right Aligned Channel 1)
// Write the ADC value directly to the DAC.
// The DAC converts this number back to a voltage on PA4.
DAC->DHR12R1 = adc_val;

// --- 3. OLED UPDATE ---
// Format and Print Resistance
snprintf(buffer, sizeof(buffer), "R: %4lu Ohms", resistance);
OLED_Print(buffer, 0, 0); // Print on Page 0 (Top line)

// Format and Print Mode (Source)
if(measurement_mode == MODE_FUNC_GEN) {
    OLED_Print("Src: FunGen   ", 2, 0); // Page 2
} else {
    OLED_Print("Src: 555Timer ", 2, 0); // Page 2
}

// --- 4. FREQUENCY DISPLAY ---
```

```
        // Split the integer (123456) into whole (1234) and decimal (56) parts
        uint32_t whole_part = frequency_x100 / 100;
        uint32_t decimal_part = frequency_x100 % 100;

        snprintf(buffer, sizeof(buffer), "F: %4lu.%02lu Hz  ", whole_part, decimal_part);
        OLED_Print(buffer, 4, 0); // Page 4

        // Small delay to prevent the screen from refreshing too fast (prevents flicker)
        Delay(500000);
    }
}

// INTERRUPT HANDLERS
/**
 * @brief  EXTI0_1_IRQHandler handles interrupts for Pins 0 and 1.
 * We use this for the USER Button on PA0.
 * "EXTI" stands for External Interrupt.
 */
void EXTI0_1_IRQHandler(void)
{
    // PR (Pending Register): This register has flags for which interrupt triggered.
    // Check if Line 0 (PA0) caused the interrupt
    if (EXTI->PR & EXTI_PR_PR0)
    {
        // Software Debounce: Wait a small amount of time to ignore switch bounce
```

```
for(volatile int i=0; i<50000; i++);

// IDR (Input Data Register): Read the actual voltage on the pin now.
// Check if Button (PA0) is still High (Pressed) after the delay.
if(GPIOC->IDR & GPIO_IDR_0)
{
  // Clear pending interrupts for Timer Pins (2 and 3)
  // This prevents a  interrupt from firing immediately after switching modes
  // We write '1' to the PR bits to clear them (Standard STM32 behavior).
  EXTI->PR = EXTI_PR_PR2 | EXTI_PR_PR3;

  // Toggle Mode Logic
  if(measurement_mode == MODE_FUNC_GEN) {
     measurement_mode = MODE_555_TIMER;

     // IMR (Interrupt Mask Register): Controls if an interrupt is "listened to".
     // 0 = Masked (Ignored/Disabled), 1 = Unmasked (Enabled)
     // We DISABLE the Function Generator interrupt (MR2)
     EXTI->IMR &= ~(EXTI_IMR_MR2);
     // We ENABLE the 555 Timer interrupt (MR3)
     EXTI->IMR |= EXTI_IMR_MR3;
  } else {
     measurement_mode = MODE_FUNC_GEN;
     // Enable FuncGen (MR2), Disable 555 (MR3)
     EXTI->IMR |= EXTI_IMR_MR2;
```

```
            EXTI->IMR &= ~(EXTI_IMR_MR3);
        }

        // Reset calculation variables so the display doesn't show old data
        capture_state = 0;
        frequency_x100 = 0;
      }
      // Clear the Pending Register bit for Line 0 so interrupt doesn't loop forever
      EXTI->PR |= EXTI_PR_PR0;
    }
}

/*
 * @brief  EXTI2_3_IRQHandler handles interrupts for Pins 2 and 3.
 * Used for Frequency Measurement (PB2 and PB3).
 */
void EXTI2_3_IRQHandler(void)
{
    // Read current Timer Count (CNT register stores the current time value)
    uint32_t current_cnt = TIM2->CNT;
    uint32_t delta = 0;
    uint8_t valid_edge = 0;

    // Check which pin triggered the interrupt and if we are in the correct mode
    if ((EXTI->PR & EXTI_PR_PR2) && (measurement_mode == MODE_FUNC_GEN)) {
```

```
    valid_edge = 1;
    EXTI->PR |= EXTI_PR_PR2; // Clear flag for Pin 2
}
else if ((EXTI->PR & EXTI_PR_PR3) && (measurement_mode == MODE_555_TIMER)) {
    valid_edge = 1;
    EXTI->PR |= EXTI_PR_PR3; // Clear flag for Pin 3
}
else {
    // If a spurious interrupt happened (wrong mode), just clear flags to be safe
    EXTI->PR |= EXTI_PR_PR2 | EXTI_PR_PR3;
}

if (valid_edge) {

    // State Machine for Frequency Capture
    if (capture_state == 0) {
        // 1st Rising Edge detected: Save current time
        last_capture_val = current_cnt;
        capture_state = 1; // Move to next state
    } else {
        // 2nd Rising Edge detected: Calculate difference (Delta)
        // Delta = Time 2 - Time 1.
        // Note: TIM2 is a 32-bit counter. If it overflows (wraps around),
        // unsigned integer math (uint32_t) automatically handles it correctly.
        delta = current_cnt - last_capture_val;
```

```
            if(delta > 0) {
                // Math: Frequency = Clock_Speed / Delta_Time
                // We multiply Clock by 100 first to keep 2 decimal places of precision.
                // cast to (uint64_t) forces 64-bit math to prevent overflow during the multiplication
step.
                frequency_x100 = (uint32_t)(((uint64_t)TIMER_FREQ_HZ * 100) / delta);
            }
            capture_state = 0; // Reset state to wait for next 1st edge
        }
    }
}
// INITIALIZATION FUNCTIONS
/**
 * @brief  Configures System Clock to 48MHz using PLL.
 *
 * 1. HSI (High Speed Internal): The default internal RC oscillator clock (8MHz).
 * 2. PLL (Phase Locked Loop): A hardware multiplier that boosts frequency.
 *
 * WHAT THIS DOES:
 * Takes the 8MHz HSI signal, divides it by 2 (4MHz), and then multiplies it
 * by 12 to achieve a stable 48MHz system heartbeat.
 *
 * EFFECT ON SYSTEM:
 * Without this, the CPU runs at 8MHz. With it, it runs 6x faster (48MHz).
 */
```

```
void SystemClock48MHz(void)
{
    // ACR (Access Control Register): Set Flash latency to 1 wait state.
    // The Flash memory is slower than the CPU at 48MHz, so we tell the CPU
    // to wait 1 cycle when reading Flash to prevent data corruption.
    FLASH->ACR |= FLASH_ACR_LATENCY;

    // Turn off PLL (Phase Locked Loop) before configuring it
    RCC->CR &= ~RCC_CR_PLLON;
    while((RCC->CR & RCC_CR_PLLRDY) != 0); // Wait for PLL to stop

    // CFGR (Configuration Register):
    // 1. Clear PLL bits
    // 2. Set PLL Source (PLLSRC) to HSI/2 (8MHz / 2 = 4MHz)
    // 3. Set PLL Multiplier (PLLMUL) to 12 (4MHz * 12 = 48MHz)
    RCC->CFGR &= ~(RCC_CFGR_PLLMUL | RCC_CFGR_PLLSRC);
    RCC->CFGR |= (RCC_CFGR_PLLMUL12);

    // Enable PLL
    RCC->CR |= RCC_CR_PLLON;
    while ((RCC->CR & RCC_CR_PLLRDY) == 0); // Wait for PLL Ready flag

    // Switch System Clock Source (SW) to PLL
    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
```

```
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL); // Wait for switch
to complete

}


/**

 * @brief  Enables clocks and sets modes for GPIO Pins.

 *

 * 1. AHBENR (AHB Peripheral Clock Enable Register):

 * The "Main Breaker" for GPIO ports. Without this, pins are dead.

 *

 * 2. MODER (Mode Register): Assigns a role to each pin.

 * - Input (00): Listens to voltage. (For Frequency signal)

 * - Output (01): Drives voltage. (For LEDs, OLED control)

 * - Alternate Function (10): Handed off to peripheral (SPI).

 * - Analog (11): Handed off to ADC/DAC.

 *

 * 3. PUPDR (Pull-Up/Pull-Down Register):

 * - Sets a default state for Input pins to prevent floating signals (noise).

 * - Pull-Down connects the pin to Ground via resistor when no signal is present.

 */

void Init_GPIO(void)

{

    // Enable clocks for GPIO Ports A, B, and C.

    RCC->AHBENR |= RCC_AHBENR_GPIOAEN | RCC_AHBENR_GPIOBEN |
RCC_AHBENR_GPIOCEN;
```

```
// --- PA1 & PA4: Analog Mode ---
// PA1 -> Connected to Potentiometer (ADC Input)
// PA4 -> Connected to Optocoupler (DAC Output)
// We write '11' (Analog) to the register positions for Pin 1 and Pin 4.
GPIOA->MODER |= (3 << (1 * 2)) | (3 << (4 * 2));

// --- PB2 & PB3: Frequency Inputs ---
// PB2 -> Function Generator Input
// PB3 -> 555 Timer Input
// 1. Clear bits to '00' (Input Mode).
GPIOB->MODER &= ~(3 << (2 * 2));
GPIOB->MODER &= ~(3 << (3 * 2));
// 2. Enable Pull-Down Resistors ('10').
// CRITICAL: If the 555 timer wire is loose, the pin will be pulled to 0V
// instead of floating and counting random noise as frequency.
GPIOB->PUPDR |= (2 << (2 * 2)) | (2 << (3 * 2));

// --- PB8, PB9, PB11: OLED Control (Outputs) ---
// PB8 -> Chip Select (CS)
// PB9 -> Data/Command (DC)
// PB11 -> Reset (RES)
// We write '01' (Output) to these positions.
GPIOB->MODER |= (1 << (8 * 2)) | (1 << (9 * 2)) | (1 << (11 * 2));

// --- PB13 & PB15: SPI Communication (Alternate Function) ---
```

```
    // PB13 -> SPI Clock (SCK)
    // PB15 -> SPI MOSI (Data)
    // We write '10' (AF) to let the SPI hardware control these pins directly.
    GPIOB->MODER |= (2 << (13 * 2)) | (2 << (15 * 2));

    // --- PC8 & PC9: LEDs (Outputs) ---
    // PC8 -> Blue LED
    // PC9 -> Green LED
    // We write '01' (Output).
    GPIOC->MODER |= (1 << (8 * 2)) | (1 << (9 * 2));
}

/**
 * @brief  Initializes Analog to Digital Converter (ADC).
 *
 * 1. STM32 (ADC): An internal voltmeter on the chip.
 * 2. Potentiometer (POT): A variable resistor on the board acting as a voltage divider.
 * 3. System Effect: The ADC reads voltage (0V-3.3V) from the POT on Pin PA1
 * and converts it to a digital number (0-4095). This number determines
 * how fast we want the 555 timer to run.
 */
void Init_ADC(void)
{
    // APB2ENR: Enable ADC Clock. Connects the 48MHz signal to the ADC block.
    RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
```

```
    // Calibration Sequence (Mandatory for accuracy)

    // We must ensure the ADC is disabled (ADEN=0) before calibrating.

    if ((ADC1->CR & ADC_CR_ADEN) != 0) ADC1->CR &= (uint32_t)(~ADC_CR_ADEN); //
Ensure disabled first

    ADC1->CR |= ADC_CR_ADCAL; // Start Calibration

    while ((ADC1->CR & ADC_CR_ADCAL) != 0); // Wait for Calibration bit to clear


    // Enable ADC

    ADC1->CR |= ADC_CR_ADEN;

    while (!(ADC1->ISR & ADC_ISR_ADRDY)); // Wait for Ready flag


    // CHSELR (Channel Selection Register):

    // We select Channel 1 because it is hardwired to Pin PA1 on the chip.

    ADC1->CHSELR = ADC_CHSELR_CHSEL1;

}


/**

 * @brief  Initializes Digital to Analog Converter (DAC).

 *

 * 1. STM32 (DAC): The microcontroller outputs a voltage on Pin PA4.

 * 2. Optocoupler (4N35): This chip (also on the breadboard) takes the

 * voltage from PA4 and lights up an internal LED.

 * 3. 555 Timer (NE555): The light from the Optocoupler changes the

 * resistance in the 555 timer circuit, which changes the speed (frequency)

 * of the square wave it generates.
```

```
 */
void Init_DAC(void)
{
    // APB1ENR: Enable DAC Clock / the switch to turn it off or on
    RCC->APB1ENR |= RCC_APB1ENR_DACEN;

    // CR: Enable DAC Channel 1 (PA4).
    // Once enabled, the pin PA4 is automatically disconnected from GPIO and connected to DAC.
    DAC->CR |= DAC_CR_EN1;
}

/**
 * @brief  Initializes SPI2 for OLED communication.
 *
 * 1. SPI (Serial Peripheral Interface): A high-speed communication protocol.
 * 2. OLED Display: The screen on the board that shows our data.
 * 3. System Effect: This enables the "pipeline" used to send pixel data
 * to the screen. Without this, the screen remains blank.
 */
void Init_SPI(void)
{
    // APB1ENR: Enable SPI2 Clock so the hardware can function.
    RCC->APB1ENR |= RCC_APB1ENR_SPI2EN;

    // CR1 (Control Register 1):
```

```
    // - MSTR: Master Mode (We control the OLED, not the other way around).
    // - SSM/SSI: Software Slave Management (We control Chip Select manually via PB8).
    // - BR: Baud Rate Divisor. Controls the speed of data transmission.
    SPI2->CR1 = SPI_CR1_MSTR | SPI_CR1_SSM | SPI_CR1_SSI |
                SPI_CR1_BR_1 | SPI_CR1_BR_0;

    // CR2 (Control Register 2):
    // - DS: Data Size. We must send 8 bits (1 byte) at a time.
    //   0111 (binary) = 7, which represents 8-bit data in this register.
    // - FRXTH: FIFO Reception Threshold. Set to trigger on 8 bits.
    SPI2->CR2 |= SPI_CR2_FRXTH | (0x7 << 8);

    // Enable SPI Peripheral (Turn it on)
    SPI2->CR1 |= SPI_CR1_SPE;
}

/**
 * @brief  Initializes Timer 2 as a 1MHz counter.
 *
 * 1. TIM2 (General Purpose Timer): A high-speed digital stopwatch.
 * 2. Prescaler (PSC): A divider that slows down the 48MHz system clock.
 * 3. System Effect: By configuring this to tick exactly 1,000,000 times
 * per second (1MHz), we can measure the time between signal edges with
 * 1 microsecond precision. This allows accurate frequency calculation.
 */
```

```
void Init_TIM2(void)
{
    // APB1ENR: Enable TIM2 Clock.
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

    // PSC (Prescaler): Divides the 48MHz system clock.
    // Formula: TimerFreq = SysFreq / (PSC + 1)
    // We want 1MHz: 48MHz / (47 + 1) = 1MHz.
    TIM2->PSC = 47;

    // ARR (Auto Reload Register): The Max Value the timer counts to.
    // We set it to Max (0xFFFFFFFF) so it acts as a free-running counter that never stops.
    TIM2->ARR = 0xFFFFFFFF;

    // EGR (Event Generation Register): Write UG (Update Generation) bit.
    // CRITICAL: The Prescaler value is buffered. It only updates on an Update Event.
    // Writing to EGR forces this event immediately so our PSC=47 takes effect now.
    // Without this line, the timer would run at 48MHz, causing wrong readings.
    TIM2->EGR |= TIM_EGR_UG;

    // SR (Status Register): The UG event sets the UIF (Update Interrupt Flag).
    // We clear it here so we don't trigger an interrupt immediately.
    TIM2->SR &= ~TIM_SR_UIF;

    // CR1: Enable Counter (CEN bit). Start the stopwatch.
```

```
    TIM2->CR1 |= TIM_CR1_CEN;
}

/**
 * @brief  Configures External Interrupts (EXTI).
 *
 * 1. EXTI (External Interrupt Controller): A hardware block that watches pins.
 * 2. Inputs: Pin PA0 (Button), PB2 (Func Gen), PB3 (555 Timer).
 * 3. System Effect: Instead of "polling" (constantly checking) the pins,
 * we tell the hardware to "tap the CPU on the shoulder" (Interrupt)
 * the instant a signal goes from Low to High (Rising Edge).
 * This guarantees we capture the exact time of the signal edge.
 */
void Init_EXTI(void)
{
    // Enable System Configuration Controller Clock.
    // This block is required to map specific GPIO pins to Interrupt Lines.
    RCC->APB2ENR |= RCC_APB2ENR_SYSCFGCOMPEN;

    // SYSCFG_EXTICR1 (External Interrupt Configuration Register 1):
    // Maps PA0 to EXTI Line 0 (User Button).
    SYSCFG->EXTICR[0] &= ~(SYSCFG_EXTICR1_EXTI0); // Clear current setting
    SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI0_PA; // Set to Port A

    // Maps PB2 to EXTI Line 2 and PB3 to EXTI Line 3.
```

```
    SYSCFG->EXTICR[0] &= ~(SYSCFG_EXTICR1_EXTI2 | SYSCFG_EXTICR1_EXTI3); //
Clear

    SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI2_PB |
SYSCFG_EXTICR1_EXTI3_PB; // Set to Port B


    // IMR (Interrupt Mask Register): Controls if an interrupt is Active.

    // We Unmask (Enable) Lines 0, 2, and 3.

    EXTI->IMR |= EXTI_IMR_MR0 | EXTI_IMR_MR2 | EXTI_IMR_MR3;


    // RTSR (Rising Trigger Selection Register):

    // We want the interrupt to fire on the RISING edge (Low -> High) of the signal.

    EXTI->RTSR |= EXTI_RTSR_TR0 | EXTI_RTSR_TR2 | EXTI_RTSR_TR3;


    // NVIC (Nested Vector Interrupt Controller): The CPU's internal interrupt manager.

    // We must enable the specific Interrupt Channels so the CPU accepts the signals.

    NVIC_EnableIRQ(EXTI0_1_IRQn); // For Line 0 (Button)

    NVIC_EnableIRQ(EXTI2_3_IRQn); // For Line 2/3 (Frequency)


    // Start with Line 3 (555 Timer) Disabled (Masked).

    // We only unmask it later when the user explicitly switches to 555 mode.

    EXTI->IMR &= ~(EXTI_IMR_MR3);

}


// OLED DRIVER FUNCTIONS

/**

 * @brief  Low-level SPI Transmit Function.
```

```
 *
 * 1. SPI2 (Peripheral): The hardware block handling the serial protocol.
 * 2. DR (Data Register): The buffer where we put the byte to be sent.
 * 3. SR (Status Register): Flags that tell us if the hardware is busy or ready.
 *
 *
 */
void SPI_Write(uint8_t data)
{
    // Wait until Transmit Buffer is Empty (TXE flag)
    while (!(SPI2->SR & SPI_SR_TXE));
    // Send Data (Casting to uint8_t ensures 8-bit write access)
    *(volatile uint8_t*)&SPI2->DR = data;
    // Wait until SPI is not Busy (BSY flag) before continuing
    while (SPI2->SR & SPI_SR_BSY);
}

void OLED_WriteCmd(uint8_t cmd)
{
    GPIOB->BRR = OLED_CS_PIN;  // CS Low (Select Display)
    GPIOB->BRR = OLED_DC_PIN;  // DC Low (Indicates this byte is a Command)
    SPI_Write(cmd);
    GPIOB->BSRR = OLED_CS_PIN; // CS High (Deselect Display)
}
```

```
void OLED_WriteData(uint8_t data)
{
    GPIOB->BRR = OLED_CS_PIN;  // CS Low
    GPIOB->BSRR = OLED_DC_PIN; // DC High (Indicates this byte is Data/Pixel info)
    SPI_Write(data);
    GPIOB->BSRR = OLED_CS_PIN; // CS High
}

void OLED_Init(void)
{
    // Hardware Reset Sequence
    // We toggle the Reset pin High-Low-High to reset the OLED controller.
    GPIOB->BSRR = OLED_RES_PIN; Delay(1000);
    GPIOB->BRR  = OLED_RES_PIN; Delay(1000);
    GPIOB->BSRR = OLED_RES_PIN; Delay(1000);

    // Initialization Commands for SSD1306
    // These commands configure the OLED internal settings (voltage pump, addressing mode,
etc.)
    OLED_WriteCmd(0xAE); // Display Off
    OLED_WriteCmd(0x20); OLED_WriteCmd(0x10); // Page Addressing Mode
    OLED_WriteCmd(0xB0); // Page Start Address

    // --- Flipped Orientation Settings ---
    // 0xC0: COM Output Scan Direction (Flipped)
    // 0xA0: Segment Re-map (Flipped)
```

```
    OLED_WriteCmd(0xC0);
    OLED_WriteCmd(0x00); // Low Column Address
    OLED_WriteCmd(0x10); // High Column Address
    OLED_WriteCmd(0x40); // Start Line Address
    OLED_WriteCmd(0x81); OLED_WriteCmd(0xFF); // Contrast Control
    OLED_WriteCmd(0xA0);

    OLED_WriteCmd(0xA6); // Normal Display (Non-inverted colors)
    OLED_WriteCmd(0xA8); OLED_WriteCmd(0x3F); // Multiplex Ratio
    OLED_WriteCmd(0xA4); // Output follows RAM content
    OLED_WriteCmd(0xD3); OLED_WriteCmd(0x00); // Display Offset
    OLED_WriteCmd(0xD5); OLED_WriteCmd(0xF0); // Clock Divide Ratio
    OLED_WriteCmd(0xD9); OLED_WriteCmd(0x22); // Pre-charge Period
    OLED_WriteCmd(0xDA); OLED_WriteCmd(0x12); // COM Pins Config
    OLED_WriteCmd(0xDB); OLED_WriteCmd(0x20); // VCOMH Deselect Level
    OLED_WriteCmd(0x8D); OLED_WriteCmd(0x14); // Charge Pump Enable (Required for
3.3V)
    OLED_WriteCmd(0xAF); // Display On
}

void OLED_Clear(void)
{
    // Loop through all 8 Pages (Rows of 8 pixels height)
    for (uint8_t i = 0; i < 8; i++) {
        OLED_WriteCmd(0xB0 + i); // Set Page Address
        OLED_WriteCmd(0x00);      // Set Lower Column Address
```

```
        OLED_WriteCmd(0x10);      // Set Higher Column Address
        // Loop through all 128 Columns and write 0x00 (Blank)
        for (uint8_t j = 0; j < 128; j++) {
            OLED_WriteData(0x00);
        }
    }
}

void OLED_Print(char *str, uint8_t page, uint8_t col)
{
    // Apply Offset to shift text right (Fixes cut-off characters on some displays)
    col += COL_OFFSET;

    OLED_WriteCmd(0xB0 + page);
    OLED_WriteCmd(0x00 + (col & 0x0F)); // Set Lower Column Start Address
    OLED_WriteCmd(0x10 + ((col >> 4) & 0x0F)); // Set Higher Column Start Address

    while (*str)
    {
        uint8_t c = *str;
        if (c < ' ' || c > 'z') c = ' '; // Safety check for font array bounds

        const uint8_t *char_ptr = Font8x8[c]; // Get font data for character

        // Send 8 bytes (one for each column of the character)
```

```
        for (uint8_t i = 0; i < 8; i++) {
            OLED_WriteData(char_ptr[i]);
        }
        str++; // Move to next character
    }
}

// Simple busy-wait delay loop
void Delay(uint32_t count)
{
    while(count--) { __asm("nop"); } // NOP = No Operation (wastes 1 cycle)
}
```

Appendix B - Hardware Connections and Wiring Guide

This appendix documents the wiring setup used to interface the following external circuit components with one another: 555 timer, potentiometer, OLED display, and optocoupler. The mentioned connections have to be established to reproduce the system behavior.

Microcontroller Pin Assignments

STM32F0	Signal	DIRECTION
PA0	USER PUSH BUTTON	INPUT
PA1	ADC	INPUT (Analog)
PA4	DAC	OUTPUT(Analog)
PB2	FUNCTION GENERATOR	INPUT

PB3	**555 TIMER**	**INPUT**
PB8	**CS#(DISAPLAY “CHIP SELECT”**	**OUTPUT**
PB9	**D/C(DISPLAY “DATA/COMMAND”)**	**OUTPUT**
PB11	**RES#(DISPLAY “RESET”)**	**OUTPUT**
PB13	**SCLK (Display D0: “Serial Clock = SPI SCK)**	**OUTPUT (AF0)**
PB15	**SCLK (Display D1: "Serial Data” = SPI MOSI)**	**OUTPUT(AF0)**
PC8	**BLUE LED**	**OUTPUT**
PC9	**GREEN LED**	**OUTPUT**

This configuration must be exactly the same to be certain that it works.

Appendix C — Hardware Setup Photograph

In the following figure C.1, the total hardware setup that has been utilized in the process of implementing and testing the developed frequency measurement system can be seen. In this setup, the STM32F0 Discovery board, the 555 timer circuit that is wired on the breadboard, the 4N35 optocoupler, and finally the OLED display and wiring that has been used in the process of testing are included. Fig. C.1: Total hardware setup of the project that has been utilized in testing of the developed frequency measurement system.

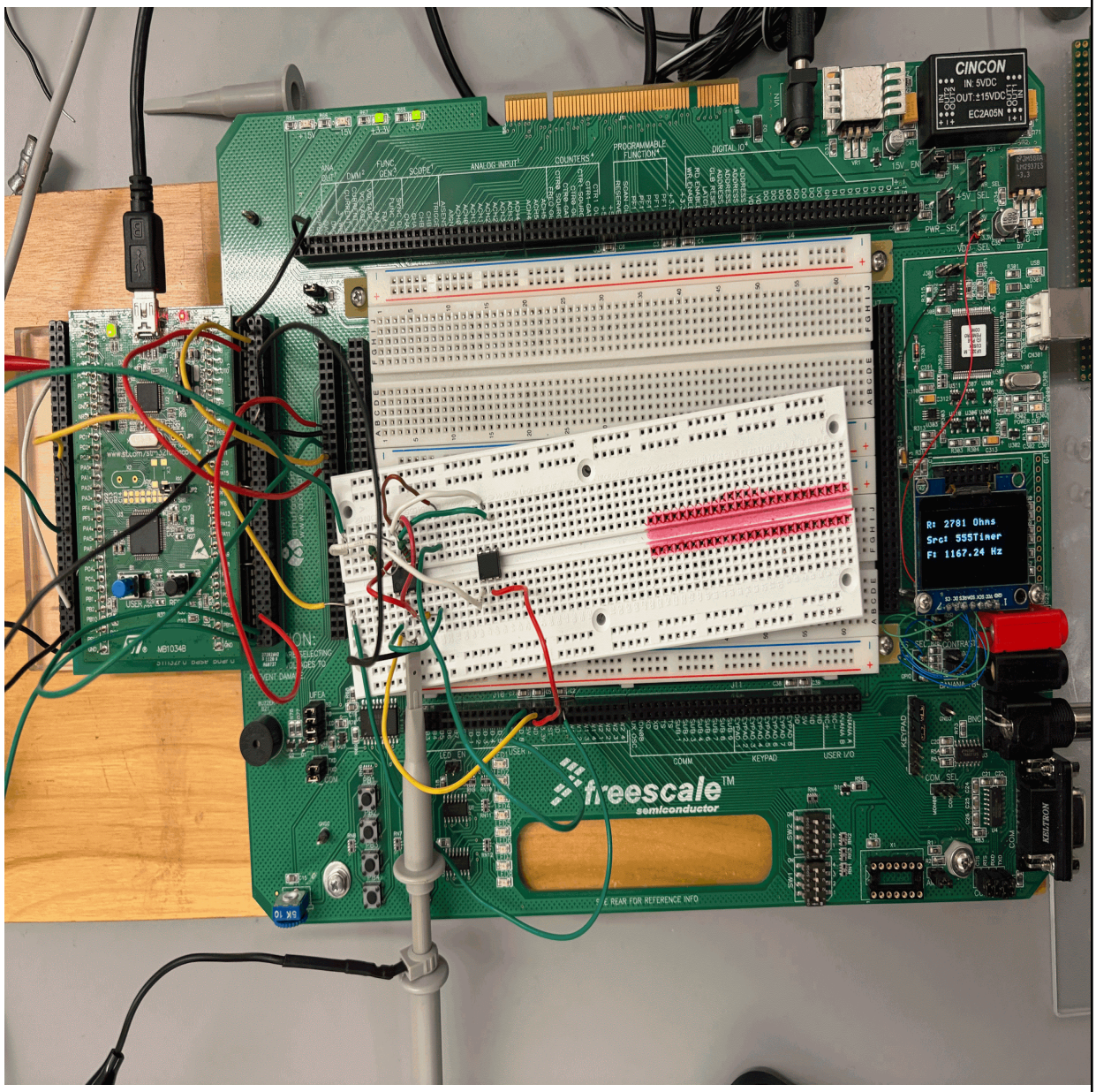

Figure C.1 — Full hardware setup used during development and testing.

References.

[1] B. Sirna and D. Rakhmatov, ECE 355: Microprocessor-Based Systems Laboratory Manual, University of Victoria, 2025.
Primary source used for all project requirements, STM32F0 peripheral configuration guidelines, and lab development procedures.

[2] University of Victoria. ECE 355 — Interface Example. UVic ECE Dept., Accessed Nov. 28, 2025. Available: https://www.ece.uvic.ca/~ece355/lab/interfacex.pdf

[3] University of Victoria, ECE 355 Laboratory Website, available at:
https://www.ece.uvic.ca/~ece355/lab/
Accessed 2025.
Used for supplementary documentation, example code snippets, and clarification of lab instructions.